AI LAB 5

Experiment 17: ID3 Decision Tree for Loan Approval Classification

AIM: To implement automated loan eligibility assessment in banking using python

QUESTION

A bank receives loan applications from customers with different financial backgrounds. A decision-support system can analyse customer information, such as income, employment status, credit history and existing loans, to classify applications as approved or rejected.

Develop a Python program that implements the ID3 algorithm using a customer loan dataset. The program should:

1. Create a dataset containing customer financial attributes.
2. Calculate entropy and information gain for each attribute.
3. Select the attribute with the highest information gain.
4. Construct a decision tree recursively.
5. Classify new loan applications using the generated decision tree.

Code:

```
import pandas as pd
import math
from collections import Counter

# Step 1: Create Customer Loan Dataset
data = {
    'Income': [
        'High', 'High', 'Medium', 'Low', 'Low',
        'Low', 'Medium', 'High', 'Medium', 'Low',
        'High', 'Medium', 'Low', 'High'
    ],
    'Employment': [
        'Permanent', 'Permanent', 'Permanent', 'Self',
        'Permanent', 'Self', 'Self', 'Self', 'Permanent',
        'Permanent', 'Self', 'Permanent', 'Self', 'Permanent'
```

```
    ],
    'Credit_History': [
        'Good', 'Good', 'Good', 'Good', 'Poor',
        'Poor', 'Good', 'Poor', 'Good', 'Poor',
        'Good', 'Poor', 'Poor', 'Good'
    ],
    'Existing_Loan': [
        'No', 'Yes', 'No', 'No', 'No',
        'Yes', 'Yes', 'No', 'No', 'Yes',
        'No', 'Yes', 'No', 'No'
    ],
    'Loan_Status': [
        'Approved', 'Approved', 'Approved', 'Approved',
        'Rejected', 'Rejected', 'Approved', 'Rejected',
        'Approved', 'Rejected', 'Approved', 'Rejected',
        'Rejected', 'Approved'
    ]
}

df = pd.DataFrame(data)

print("CUSTOMER LOAN DATASET")
print(df)

# Step 2: Calculate Entropy
def entropy(target):
    counts = Counter(target)
    total = len(target)

    result = 0
    for count in counts.values():
        probability = count / total
        result -= probability * math.log2(probability)

    return result

# Step 3: Calculate Information Gain
def information_gain(dataset, attribute, target):
    total_entropy = entropy(dataset[target])
    weighted_entropy = 0
```

```
    for value in dataset[attribute].unique():
        subset = dataset[dataset[attribute] == value]
        weight = len(subset) / len(dataset)

        weighted_entropy += weight * entropy(subset[target])

    return total_entropy - weighted_entropy

# Step 4: Select Best Attribute
def best_attribute(dataset, attributes, target):
    gains = {}

    for attribute in attributes:
        gains[attribute] = information_gain(
            dataset, attribute, target
        )

    print("\nInformation Gain:")
    for attribute, gain in gains.items():
        print(f"{attribute}: {gain:.4f}")

    return max(gains, key=gains.get)

# Step 5: Build ID3 Decision Tree Recursively
def build_tree(dataset, attributes, target):

    # If all records belong to one class
    if len(dataset[target].unique()) == 1:
        return dataset[target].iloc[0]

    # If no attributes remain
    if not attributes:
        return dataset[target].mode()[0]

    # Select attribute with highest information gain
    best = best_attribute(dataset, attributes, target)

    tree = {best: {}}
```

```
    remaining_attributes = [
        attr for attr in attributes if attr != best
    ]

    # Create branches recursively
    for value in dataset[best].unique():

        subset = dataset[dataset[best] == value]

        if subset.empty:
            tree[best][value] = dataset[target].mode()[0]
        else:
            tree[best][value] = build_tree(
                subset,
                remaining_attributes,
                target
            )

    return tree

# Step 6: Prediction Using Decision Tree
def predict(tree, sample):

    if not isinstance(tree, dict):
        return tree

    attribute = next(iter(tree))
    value = sample.get(attribute)

    if value not in tree[attribute]:
        return "Unknown"

    return predict(tree[attribute][value], sample)

# Step 7: Train the Decision Tree
attributes = [
    'Income',
    'Employment',
    'Credit_History',
    'Existing_Loan'
```

```
]

target = 'Loan_Status'

print("\nINITIAL ENTROPY:",
      round(entropy(df[target]), 4))

print("\nBUILDING DECISION TREE...")

decision_tree = build_tree(df, attributes, target)

print("\nGENERATED DECISION TREE:")
print(decision_tree)

# Step 8: Test New Loan Applications
new_customers = [
    {
        'Income': 'High',
        'Employment': 'Permanent',
        'Credit_History': 'Good',
        'Existing_Loan': 'No'
    },
    {
        'Income': 'Low',
        'Employment': 'Self',
        'Credit_History': 'Poor',
        'Existing_Loan': 'Yes'
    },
    {
        'Income': 'Medium',
        'Employment': 'Permanent',
        'Credit_History': 'Poor',
        'Existing_Loan': 'Yes'
    }
]

print("\nLOAN APPROVAL PREDICTIONS:")

for i, customer in enumerate(new_customers, 1):
    result = predict(decision_tree, customer)
```

```
    print(f"\nCustomer {i}:")
    print(customer)
    print("Loan Status:", result)
```

Output:

```
CUSTOMER LOAN DATASET
    Income Employment Credit_History Existing_Loan Loan_Status
0     High  Permanent           Good            No    Approved
1     High  Permanent           Good           Yes    Approved
2   Medium  Permanent           Good            No    Approved
3      Low       Self           Good            No    Approved
4      Low  Permanent           Poor            No    Rejected
5      Low       Self           Poor           Yes    Rejected
6   Medium       Self           Good           Yes    Approved
7     High       Self           Poor            No    Rejected
8   Medium  Permanent           Good            No    Approved
9      Low  Permanent           Poor           Yes    Rejected
10    High       Self           Good            No    Approved
11  Medium  Permanent           Poor           Yes    Rejected
12     Low       Self           Poor            No    Rejected
13    High  Permanent           Good            No    Approved

INITIAL ENTROPY: 0.9852

BUILDING DECISION TREE...

Information Gain:
Income: 0.2378
Employment: 0.0113
Credit_History: 0.9852
Existing_Loan: 0.0481

GENERATED DECISION TREE:
{'Credit_History': {'Good': 'Approved', 'Poor': 'Rejected'}}

LOAN APPROVAL PREDICTIONS:

Customer 1:
{'Income': 'High', 'Employment': 'Permanent', 'Credit_History': 'Good',
'Existing_Loan': 'No'}
Loan Status: Approved

Customer 2:
{'Income': 'Low', 'Employment': 'Self', 'Credit_History': 'Poor',
'Existing_Loan': 'Yes'}
Loan Status: Rejected
```

```
Customer 3:
{'Income': 'Medium', 'Employment': 'Permanent', 'Credit_History': 'Poor',
'Existing_Loan': 'Yes'}
Loan Status: Rejected
```

Experiment 18: Monkey and Banana Problem Using State-Space Planning

AIM: To implement Goal-based planning in an autonomous warehouse robot using python.

QUESTION

A monkey is inside a room containing a box and a banana hanging from the ceiling. The monkey cannot reach the banana directly. It must move to the box, push it beneath the banana, climb onto the box and grasp the banana.

Develop a Python program that represents the problem using state-space representation and determines the sequence of actions required to obtain the banana.

Code:

```
# Experiment 18: Monkey and Banana Problem Using State-Space Planning

# Initial State
# (Monkey Position, Box Position, Monkey Status, Has Banana)

initial_state = ("Door", "Window", "OnFloor", False)

# Goal State
goal_state = ("Middle", "Middle", "OnBox", True)


def monkey_banana_problem():
    state = initial_state

    print("Initial State:", state)
    print("\nActions performed:\n")

    # Step 1: Monkey moves to the box
    state = ("Window", "Window", "OnFloor", False)
    print("1. Monkey moves from Door to Window (near the box).")

    # Step 2: Monkey pushes the box below the banana
```

```
    state = ("Middle", "Middle", "OnFloor", False)
    print("2. Monkey pushes the box to the Middle (below the banana).")

    # Step 3: Monkey climbs onto the box
    state = ("Middle", "Middle", "OnBox", False)
    print("3. Monkey climbs onto the box.")

    # Step 4: Monkey grasps the banana
    state = ("Middle", "Middle", "OnBox", True)
    print("4. Monkey grasps the banana.")

    print("\nFinal State:", state)

    if state == goal_state:
        print("\nGoal Achieved! Monkey successfully got the banana.")
    else:
        print("\nGoal Not Achieved.")


# Execute the program
monkey_banana_problem()
```

Output:

```
Initial State: ('Door', 'Window', 'OnFloor', False)

Actions performed:

1. Monkey moves from Door to Window (near the box).
2. Monkey pushes the box to the Middle (below the banana).
3. Monkey climbs onto the box.
4. Monkey grasps the banana.

Final State: ('Middle', 'Middle', 'OnBox', True)

Goal Achieved! Monkey successfully got the banana.
```

Experiment 19: Towers of Hanoi Using Recursion in Python

AIM: To implement automated disk transfer and robotic manipulation using python.

QUESTION

A robotic system needs to transfer a stack of objects from one platform to another while following specific placement constraints. The Towers of Hanoi problem demonstrates how a system can break a complex task into smaller subproblems using recursion.

Three rods are provided: Source (A), Auxiliary (B) and Destination (C). A stack of disks of different sizes is placed on the source rod.

Develop a Python program to transfer all disks from the source rod to the destination rod while following these rules:

1. Only one disk can be moved at a time.
2. Only the top disk of a rod can be moved.
3. A larger disk cannot be placed on a smaller disk.

Code:

```
# Experiment 19: Towers of Hanoi Using Recursion

def tower_of_hanoi(n, source, auxiliary, destination):

    if n == 1:
        print("Move disk 1 from", source, "to", destination)
        return

    # Move n-1 disks from source to auxiliary
    tower_of_hanoi(n - 1, source, destination, auxiliary)

    # Move the largest disk to destination
    print("Move disk", n, "from", source, "to", destination)

    # Move n-1 disks from auxiliary to destination
```

```
    tower_of_hanoi(n - 1, auxiliary, source, destination)


# Main Program
n = int(input("Enter the number of disks: "))

print("\nSteps to solve Towers of Hanoi:\n")

tower_of_hanoi(n, 'A', 'B', 'C')

print("\nAll disks transferred successfully!")
```

Output:

```
Enter the number of disks: 3

Steps to solve Towers of Hanoi:

Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C

All disks transferred successfully!
```

Experiment 20: Simple Reflex Agent for Vacuum World

AIM: To implement autonomous cleaning robot for smart offices using python.

QUESTION:

A smart office uses an autonomous vacuum-cleaning robot to clean different rooms. The robot senses whether its current location is dirty or clean and selects an appropriate action, such as cleaning the floor or moving to another location.

Develop a Python program that simulates a vacuum-cleaning agent operating in a four-room environment.

The agent should:

1. Identify its current location.
2. Detect whether the current room is dirty or clean.
3. Clean the room if it is dirty.
4. Move to the next room if the current room is clean.
5. Stop the simulation when all rooms are clean.

Code:

```
# Experiment 20: Simple Reflex Agent for Vacuum World

# Initial state of four rooms
rooms = {
    'A': 'Dirty',
    'B': 'Dirty',
    'C': 'Clean',
    'D': 'Dirty'
}

# Initial position of the vacuum agent
current_location = 'A'

# Order of room movement
room_list = ['A', 'B', 'C', 'D']

print("Initial Room Status:", rooms)
print("\nVacuum Cleaning Agent Started\n")

# Simple Reflex Agent
for room in room_list:

    current_location = room

    print("Agent is in Room:", current_location)

    if rooms[current_location] == 'Dirty':
        print("Room is Dirty. Cleaning...")
        rooms[current_location] = 'Clean'
        print("Room", current_location, "is now Clean.")
```

```
    else:
        print("Room", current_location, "is already Clean.")

    print()

# Check whether all rooms are clean
if all(status == 'Clean' for status in rooms.values()):
    print("All rooms are clean!")
    print("Vacuum Cleaning Agent Stopped.")

print("\nFinal Room Status:", rooms)
```

Output:

```
Initial Room Status: {'A': 'Dirty', 'B': 'Dirty', 'C': 'Clean', 'D':
'Dirty'}

Vacuum Cleaning Agent Started

Agent is in Room: A
Room is Dirty. Cleaning...
Room A is now Clean.

Agent is in Room: B
Room is Dirty. Cleaning...
Room B is now Clean.

Agent is in Room: C
Room C is already Clean.

Agent is in Room: D
Room is Dirty. Cleaning...
Room D is now Clean.

All rooms are clean!
Vacuum Cleaning Agent Stopped.

Final Room Status: {'A': 'Clean', 'B': 'Clean', 'C': 'Clean', 'D':
'Clean'}
```